

createCollection & collMod

Validação de esquema em etapas: criar, testar, consultar, alterar e validar novamente.

Módulo 1: primeira coleção

Comece com poucos campos; faça o validador crescer somente após os primeiros testes.

Etapa 1: coleção 'alunos' com duas regras

1. Crie a coleção

```
db.createCollection("alunos", {
  validator: { $jsonSchema: {
    bsonType: "object",
    required: ["nome", "idade"],
    properties: {
      nome: { bsonType: "string" },
      idade: { bsonType: "int", minimum: 0 }
    }
  }}
})
```

2. Teste imediatamente

```
db.alunos.insertOne({
  nome: "Ana", idade: NumberInt(21)
}) // válido

db.alunos.insertOne({
  nome: "Bruno", idade: "21"
}) // inválido: idade é string
```

Módulo 2: consulte antes de modificar

find() revela campos ausentes, valores e tipos BSON presentes na coleção.

Etapa 2: conheça os documentos com find()

Localizar campos e valores

```
// todos os documentos
db.alunos.find()

// projeção: somente nome e idade
db.alunos.find({}, { nome: 1, idade: 1 })

// documentos sem o campo idade
db.alunos.find({ idade: { $exists: false } })
```

Inspecionar características

```
// idade armazenada como int
db.alunos.find({ idade: { $type: "int" } })

// mostrar o tipo de cada idade
// Idade do tipo int
db.alunos.find({
  idade: {
    $type: "int"
  }
})
```

Módulo 3: evoluir o validador com collMod

Regra central: o novo validator substitui o anterior; envie regras antigas mantidas + regras novas.

collMod: quatro decisões antes de executar

1

Coleção

collMod: "alunos"

Escolha o alvo correto.

2

Validador completo

antigo + novo

Não envie só a mudança.

3

Nível

strict ou moderate

Defina quem será validado.

4

Ação

error ou warn

Defina bloquear ou avisar.

Se um campo antigo não aparecer no novo validator, ele deixa de ser validado.

strict ou moderate: qual é a diferença?

Os dois validam novas inserções; a diferença aparece principalmente ao atualizar documentos antigos inválidos.

strict

Rigor máximo

- ✓ Valida todas as inserções
- ✓ Valida todas as atualizações
- ✓ Documento antigo inválido deve ficar válido ao ser atualizado

moderate

Transição gradual

- ✓ Valida todas as inserções
- ✓ Valida atualizações de documentos que já eram válidos
- ✓ Documento antigo inválido pode continuar inválido temporariamente

Regra de memória: moderate não libera novas inserções inválidas.

Demonstração: um documento legado sem 'curso'

O documento foi criado antes da nova regra e permanece armazenado após a alteração do validador.

```
// documento antigo
{
  nome: "Ana",
  idade: NumberInt(21)
  // curso ainda não existe
}
```

Tentativa de alterar somente o nome

```
db.alunos.updateOne(
  { nome: "Ana" },
  { $set: { nome: "Ana Souza" } }
)
```

strict

Bloqueia: o documento resultante continua sem curso.

moderate

Permite: o documento já era inválido antes da atualização.

Correção correta: inclua curso na atualização e torne o documento válido.

Estratégia de migração: moderate → corrigir → strict

Use moderate como etapa temporária, não como estado permanente sem acompanhamento.

1

Aplicar moderate

Novas gravações seguem a regra; dados legados ganham tempo para correção.

2

Localizar pendências

`db.alunos.find({ curso: { $exists: false } })`

3

Corrigir documentos

Atualize os registros e preencha os campos exigidos.

4

Confirmar a limpeza

A consulta de pendências deve retornar zero documentos.

5

Migrar para strict

Reenvie o validator completo com `validationLevel: "strict"`.

Level e Action respondem a perguntas diferentes

Combine os dois parâmetros: um define quando validar; o outro define a consequência da violação.

validationLevel

Quem será fiscalizado?

strict

Todas as inserções e atualizações.

moderate

Inserções e atualizações de documentos que já eram válidos.

validationAction

O que acontece se houver violação?

error

Bloqueia a gravação.

warn

Permite a gravação e registra uma advertência no log.

Level escolhe quem é validado; Action escolhe a consequência.

As quatro combinações possíveis

Não existe conflito entre moderate e error: essa combinação é especialmente útil em migrações graduais.

strict + error

Valida amplamente
Bloqueia violações

strict + warn

Valida amplamente
Permite e avisa

moderate + error

Protege novas gravações
Tolera legado; bloqueia quando aplicável

moderate + warn

Transição mais permissiva
Tolera e registra advertências

Migração recomendada: moderate + error → corrigir legado → strict + error

Exercício: preveja, execute e explique

Para cada cenário, registre primeiro sua previsão: BLOQUEIA, PERMITE ou PERMITE COM AVISO.

```
// use o mesmo validator de alunos
// curso é obrigatório
db.runCommand({
  collMod: "alunos",
  validator: { $jsonSchema: { /* schema completo */ },
  validationLevel: "moderate",
  validationAction: "error"
})
```

Teste os quatro cenários

1. Inserir Bruno sem curso.
2. Alterar somente o nome de Ana, que já era inválida.
3. Alterar um aluno válido e remover curso.
4. Trocar error por warn e repetir o cenário 1.

Gabarito esperado: 1 BLOQUEIA • 2 PERMITE • 3 BLOQUEIA • 4 PERMITE COM AVISO

Módulo 4: laboratório progressivo

Três pequenas alterações: adicionar campo, mudar o comportamento e auditar antes de endurecer.

Etapa 3: acrescenta 'curso' sem perder as regras anteriores

```
db.runCommand({
  collMod: "alunos",
  validator: { $jsonSchema: {
    bsonType: "object",
    required: ["nome", "idade", "curso"],
    properties: {
      nome: { bsonType: "string" },    // regra mantida
      idade: { bsonType: "int", minimum: 0 }, // regra mantida
      curso: { bsonType: "string" }    // regra nova
    }
  }},
  validationLevel: "strict",
  validationAction: "error"
})
```

Antes e depois

1. Execute o collMod.
2. Insira um aluno sem curso.
3. Observe o bloqueio.
4. Insira nome + idade + curso.
5. Consulte com find().

A estrutura inteira foi reenviada.

Desafio em duplas: três coleções pequenas



Ex. 1 — alunos

Use collMod para tornar email obrigatório. Preserve nome, idade e curso. Teste uma inserção válida e uma inválida.



Ex. 2 — produtos

Crie produtos com nome string, estoque int ≥ 0 e preco double ≥ 0 . Use find() para localizar estoque do tipo int.



Ex. 3 — pedidos

Crie pedidos com cliente como object e itens como array. Depois acrescente status usando collMod sem perder as regras anteriores.

Síntese & discussão

*Uma alteração segura segue sempre o mesmo ciclo: consultar → escrever o schema completo → executar collMod
→ testar válido e inválido → consultar novamente.*

find() → schema completo → collMod → testes → find()

Checklist para não perder regras

Antes do collMod

- ✓ confirmar o nome da coleção
- ✓ consultar documentos existentes com find()
- ✓ copiar o validator atual completo
- ✓ somar regras antigas e novas
- ✓ escolher strict ou moderate
- ✓ escolher error ou warn
- ✓ testar uma inserção válida e outra inválida

Referências oficiais

MongoDB Manual

- Schema Validation
- comando collMod
- consultas com \$jsonSchema

As referências completas estão nas notas do slide.